Model Selection

Post-Inference Problem

In previous classes

You learned

- how to select a model using *stepwise* algorithms and *regularized* models

- how to use an estimated model to make inference about the population

Can we use the model selected by stepwise or regularized methods to make inference?

The post-inference problem

Beyond variable selection, there are many aspects involved in selecting a model and a method to estimate it, for example,

- Do we want a parametric or non-parametric approach?
- Do we want to assume a functional form for the relationship between Y and $\mathbf{X}$ (e.g., linear, quadratic, exponential)?
- Prediction vs inference
- Is model interpretability important?
- Which variables should be included in the model?

Simulation Study

If we use data to select a model, can we fit the selected model using the same data to make inference??

To examine this problem, we'll use a **simulation study** so that

- you know the values of parameters you are trying to infer
- you can quantify errors of your inferential procedure due to randomness
- you can repeat the same scenario multiple times to quantify error rates

Simulation Design

Scroll down to see full content

1. Setting (variables): a response variable Y and $p = 10$ covariates with Normal distribution

2. Setting (coefficients): set all coefficients to 0, so none of the generated covariates will have an effect on Y

> we expect the intercept only model to be better than any LR that includes any of the covariates

3. Generate 100 observations (n) for this setting

4. Apply the forward selection algorithm to select at most 3 variables among the 10 available, using the adj R^2

5. Use the selected model and the same dataset to make inference about the population

6. Replicate this study 1,000 times (`rep`) and compute the type I error rate.

Generating `data`

In the worksheet, we included a function that generates all the data **at once**!

> `data` is a tibble with 100000 (100 x 1000) rows and 12 (10 covariates + 1 response + 1 rep ID) columns

- `data` should be treated as 1000 datasets of size 100
- the variable `replicate` identifies each dataset

```
dim(dataset)
```

```
[1] 100000     12
```

```
length(unique(dataset$replicate))
```

```
[1] 1000
```

```
head(dataset,2) %>% knitr::kable()
```

X1	X2	X3	X4	X5	X6	X7	X8	X9	X10	Y	replicate
8.07	8.86	15.62	3.37	2.79	0.77	5.65	-8.65	7.52	9.39	-6.99	1
-3.96	3.59	15.84	3.05	1.38	-0.01	-14.54	7.11	-9.83	-3.03	21.06	1

```
tail(dataset,2) %>% knitr::kable()
```

X1	X2	X3	X4	X5	X6	X7	X8	X9	X10	Y	replicate
6.97	15.02	6.46	-3.43	13.39	6.20	-14.12	-11.21	15.91	15.14	10.76	1000
-6.55	-1.01	6.46	0.97	-0.33	2.86	10.33	-17.67	4.20	2.07	0.28	1000

Selection Function

We created a function for you:

- takes a dataset as an input
- runs the forward selection algorithm and selects at most 3 variables using adj R^2
- fits LS on the selected variables

```r
forward_selection_function <- function(dataset){
    sel_model <- regsubsets(x = Y ~ .,
                      nvmax = 3,
                      data = dataset,
                      method = "forward",
)

sel_model_summary <- summary(sel_model)

adj_r2_min = which.max(sel_model_summary$adjr2)

selected_var <- names(coef(sel_model, adj_r2_min))[-1]

data_subset <- dataset %>%
  select(all_of(selected_var),Y)

selected_model <- lm(Y ~ .,data = data_subset)

```

For example

Using the selection function in the first dataset

```
selected_model_LS <- forward_selection_function(full_models$data[[1]])

selected_model_LS%>%
  tidy() %>%
  mutate_if(is.numeric, round, 3) %>%
  knitr::kable()
```

term	estimate	std.error	statistic	p.value
(Intercept)	4.325	1.354	3.194	0.002
X4	0.154	0.106	1.456	0.149
X10	-0.110	0.106	-1.039	0.301

> `full_models` has 3 objects, one is a list of datasets. You will use `map` to pass this function to the whole list!!

The test

We can use the test in `glance` to compare the selected model versus the intercept-only model

```
glance(selected_model_LS) %>%
             pull(p.value) %>%
             round(4)
```

```
 value
0.2427
```

In this case, we fail to reject the null hypothesis (as expected!)

Recall that all coefficients are 0!

> You will use `map` to pass this function to all selected models!!

```
# A tibble: 10 × 4
# Groups:   replicate [10]
   replicate data                fs_model F_pvalue
       <int> <list>               <list>      <dbl>
 1         1 <tibble [100 × 11]> <lm>      0.243
 2         2 <tibble [100 × 11]> <lm>      0.138
 3         3 <tibble [100 × 11]> <lm>      0.0320
 4         4 <tibble [100 × 11]> <lm>      0.314
 5         5 <tibble [100 × 11]> <lm>      0.142
 6         6 <tibble [100 × 11]> <lm>      0.106
 7         7 <tibble [100 × 11]> <lm>      0.0701
 8         8 <tibble [100 × 11]> <lm>      0.00299
 9         9 <tibble [100 × 11]> <lm>      0.0730
10        10 <tibble [100 × 11]> <lm>      0.00823
```

Wait! In some datasets there is enougth evidence to reject the null hypothesis??


iStock™
Credit: Tempura
1160233801

A post-inference problem

The problem : we are using the **same data** to find the variables and fit the model

In the sample at hand, the adjusted R^2 improved after selecting these variables. Thus, we have a much higher chance of wrongly rejecting H_0

A possible solution: split the dataset into two parts: one to select the model and the other one for inference.

Would that solve the problem?

Splitting the data

To implement this solution, this time we are going to split each dataset into two parts:

- one part will be used to select a model, and
- the other part for inference.

> since the sample size has changed, for a fair comparison, we'll fit the selected models and test them on the first half, then recompute the error rate.

You will use the functions `map`, `map_dbl`, `map2` and `update`. Let's take a look at them.

The `map` function

We can use the function `map` to efficiently apply the same function to all datasets **at once** (although a loop is run behind scenes)!

- the function `map` applies a funtion to the list of datasets
- the output of `map` is also a list

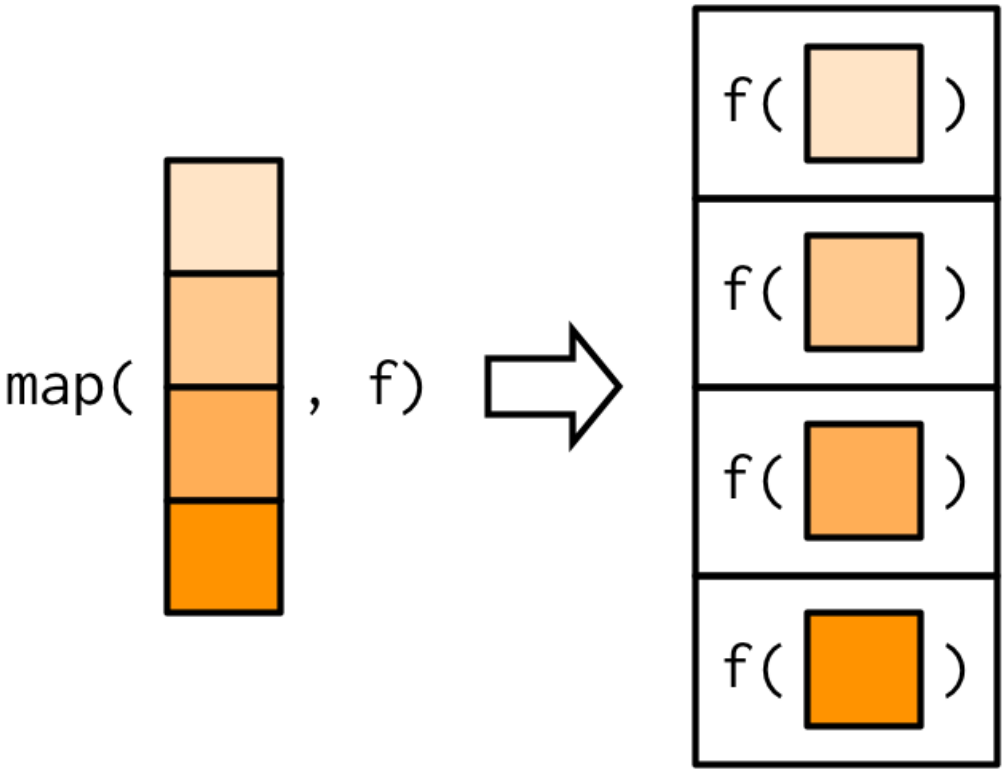

Image from Advaced R, by H. Wickham

For example (from Advaced R: 9.2)

```
1 triple <- function(x) x * 3
2
3 map(1:3, triple)
```

```
[[1]]
[1] 3

[[2]]
[1] 6

[[3]]
[1] 9
```

Note that the output of `map` is a list!

Special `map` functions

The `map` function is very general because any type of object can be stored in a list

To return a list with a simpler structure we can use its variants: `map_lgl()`, `map_int()`, `map_dbl()`, and `map_chr()`

For example (from Advaced R: 9.2.1), `map_dbl()` always returns a double vector

```
1 map_dbl(mtcars, mean)
```

```
      mpg       cyl      disp        hp      drat        wt      qsec 
20.090625  6.187500 230.721875 146.687500  3.596563  3.217250 17.848750 
       vs        am      gear      carb 
 0.437500  0.406250  3.687500  2.812500 
```

Advaced R: 9.2.2. Anonymous functions

You can write your own function directly in the second argument

```
1 map_dbl(mtcars, function(x) length(unique(x)))
```

```
mpg  cyl disp   hp drat   wt qsec   vs   am gear carb
 25    3   27   22   22   29   30    2    2    3    6
```

Advaced R: 9.2.2. Shortcuts

The shorcut ~ can be used to write only the body of the function and the first argument (`.x`) will be passed to it

```
1 map_dbl(mtcars, ~ length(unique(.x)))
```

```
mpg  cyl disp   hp drat   wt qsec   vs   am gear carb
 25    3   27   22   22   29   30    2    2    3    6
```

Example

```r
mtcars_list <- list(mtcars, mtcars)
map(mtcars_list, ~ lm(mpg ~ ., data = .x))
```

```
[[1]]

Call:
lm(formula = mpg ~ ., data = .x)

Coefficients:
(Intercept)          cyl         disp           hp         drat           wt
   12.30337     -0.11144      0.01334     -0.02148      0.78711     -3.71530
       qsec           vs           am         gear         carb
    0.82104      0.31776      2.52023      0.65541     -0.19942


[[2]]

Call:
lm(formula = mpg ~ ., data = .x)

Coefficients:
(Intercept)          cyl         disp           hp         drat           wt
   12.30337     -0.11144      0.01334     -0.02148      0.78711     -3.71530
       qsec           vs           am         gear         carb
    0.82104      0.31776      2.52023      0.65541     -0.19942
```

For example (from your worksheet)

```
full_models <-
    dataset %>%
    group_by(replicate) %>%
    nest() %>%
    mutate(models = map(.x = data, ~ lm(Y ~ ., data = .x)))`
```

- a list of datasets is created with `group_by()` and `nest`

- `map` is used to fit LS to each dataset: the function `lm` (f in the figure) is applied to each element of `data`

> the first argument `data` can be written as `.x = data` (more useful when there are 2 arguments, see `map2`)

- the output is a list containing the elements `replicate`, `data`, and `models` (the fitted models)!!

map2

It is the same as `map` but takes 2 arguments

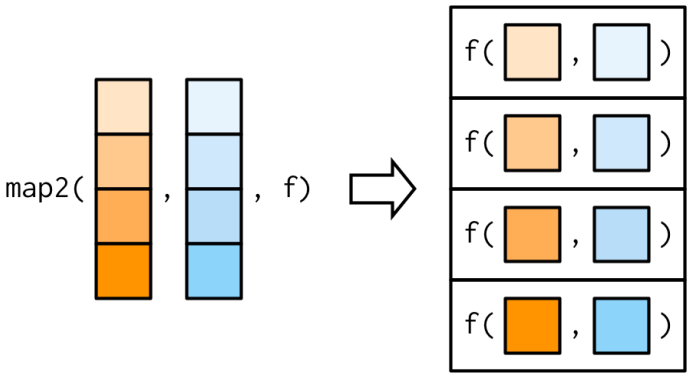

Image from 9.4.2: Advaced R, by H. Wickham

From the worksheet:

```
ls_fit = map2(.x = data, .y = lasso_selected_covariates,
                    ~lm(Y ~ ., data = .x[,c(.y, 'Y')]))
```

update

```r
# Fit an initial linear model
model1 <- lm(mpg ~ wt + hp, data = mtcars)

# Update the model to add another predictor (qsec)
model2 <- update(model1, . ~ . + qsec)

# Print the updated model
summary(model2)
```

```
Call:
lm(formula = mpg ~ wt + hp + qsec, data = mtcars)

Residuals:
    Min      1Q  Median      3Q     Max 
-3.8591 -1.6418 -0.4636  1.1940  5.6092 

Coefficients:
            Estimate Std. Error t value Pr(>|t|)    
(Intercept) 27.61053    8.41993   3.279  0.00278 ** 
wt          -4.35880    0.75270  -5.791 3.22e-06 ***
hp          -0.01782    0.01498  -1.190  0.24418    
qsec         0.51083    0.43922   1.163  0.25463    
```

Work on section 2 of worksheet_07

Summary: post-inference problem

- we can not use the same data to select variables of the model and to conduct inference (“doble dipping”).
- the inference results given by the `lm` are not valid (as seen in the first part of the worksheet).
- if we split the data, we can use one part to select and the other part to estimate and build tests.
- more sophisticated methods have been proposed to address this problem (beyond the scope of this course).